# ■ DIT Bibliothèque Numérique

## Architecture Microservices — Rapport Technique

| | |
|---|---|
| **Projet** | DevOps — Master 1 Intelligence Artificielle |
| **Institution** | Dakar Institute of Technology (DIT) |
| **Période** | 09 Mars 2026 → 29 Mars 2026 |
| **Équipe** | Babacar Niang · Zakaria Sawadogo · Souleymane Dieng · Fabrice Jordan Ramos |

React.js | Python Flask | PostgreSQ L 15 | Docker | Nginx | Jenkins CI/CD | Swagger

## 01 — Présentation du Projet

La **DIT Bibliothèque Numérique** est une application web complète de gestion de bibliothèque universitaire, développée dans le cadre du cours DevOps du Master 1 Intelligence Artificielle du Dakar Institute of Technology. Ce projet met en pratique les principes modernes du développement logiciel, de la conteneurisation et du déploiement continu.

### Objectifs du projet

- Concevoir une application distribuée en architecture microservices
- Mettre en oeuvre la conteneurisation Docker et l'orchestration via Docker Compose
- Automatiser le pipeline CI/CD avec Jenkins (lint, tests, build, deploy, health check)
- Fournir une interface utilisateur moderne en React.js avec visualisation de données
- Documenter les APIs avec Swagger/OpenAPI pour faciliter les tests et l'intégration

### Contexte académique

Ce projet a été réalisé en équipe de quatre étudiants sur une durée de trois semaines. Il simule un environnement de production réel avec séparation des responsabilités, versioning Git collaboratif (branches, pull requests) et déploiement automatisé.

## Équipe du projet

| Membre | Rôle principal |
|---|---|
| Babacar | Backend — Books & Users Services, Architecture BD |
| Zakaia | Backend — Support services, intégration données |
| Souleymane Diene | Frontend — Interface React, Dashboard, Graphiques |
| Fabrice Jordan Ramos | DevOps — Docker, Jenkins CI/CD, Swagger, Nginx |

# 02 — Architecture du Système

L'application repose sur une architecture orientée microservices, où chaque composant est isolé dans son propre conteneur Docker et communique via un réseau interne dédié. Nginx sert à la fois de serveur web pour le frontend React et de reverse proxy pour router les requêtes API vers les bons microservices.

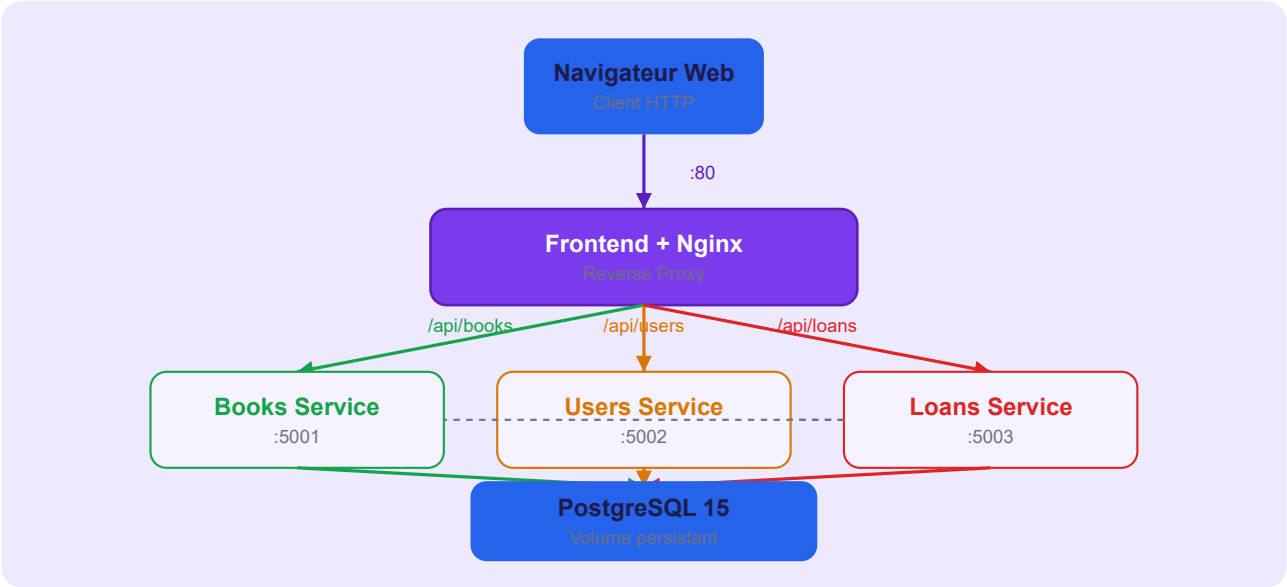



Figure 1 — Diagramme d'architecture globale du système

## Composants de l'architecture

| Composant | Technologie | Port | Rôle |
|---|---|---|---|
| Frontend | React.js + Vite + Nginx | :80 | Interface utilisateur + Reverse Proxy |
| Books Service | Python Flask + Gunicorn | :5001 | Gestion du catalogue de livres |
| Users Service | Python Flask + Gunicorn | :5002 | Gestion des membres |
| Loans Service | Python Flask + Gunicorn | :5003 | Gestion des emprunts |

| | | | |
|---|---|---|---|
| Base de données | PostgreSQL 15 (Alpine) | :5432 | Persistance des données |

### Réseau et persistance

Tous les conteneurs partagent le réseau Docker bridge **dit-network**. La base de données PostgreSQL utilise un volume nommé **dit-postgres-data** pour assurer la persistance des données entre les redémarrages. Les services backend communiquent entre eux par leur nom de conteneur DNS interne.

## 03 — Description des Microservices

Chaque microservice est autonome, possède son propre Dockerfile et est déployé avec **Gunicorn** (2 workers) pour des performances optimales. Ils partagent la même base de données PostgreSQL mais opèrent sur des tables distinctes. Chaque service intègre **Flasgger** pour exposer une documentation Swagger interactive.

### ■ Books Service

**PORT 5001**

Gestion complète du catalogue de livres — CRUD + statistiques + disponibilité

- GET /api/books
- POST /api/books
- PUT /api/books/<id>
- GET /api/books/stats

Le Books Service gère l'intégralité du catalogue. Il expose des endpoints pour lister, créer, modifier et supprimer des livres. Il maintient en temps réel le stock de copies disponibles et est interrogé par le Loans Service lors de chaque emprunt ou retour. Le modèle de données inclut : titre, auteur, ISBN unique, catégorie, nombre total de copies, copies disponibles, année de publication et description.

### ■ Users Service

**PORT 5002**

Gestion des membres — Étudiants, Professeurs, Personnel administratif

- GET /api/users
- POST /api/users
- POST /api/users/login
- GET /api/users/stats

Le Users Service gère les membres de la bibliothèque avec trois profils : Étudiant, Professeur et Personnel administratif. Il inclut un système d'authentification basique avec hachage des mots de passe (Werkzeug). Le filtrage par type d'utilisateur est disponible sur le endpoint principal, et des statistiques de répartition sont exposées pour alimenter les graphiques du tableau de bord.

### ■ Loans Service

**PORT 5003**

Gestion des emprunts — Orchestrateur inter-services (14 jours par défaut)

- GET /api/loans
- POST /api/loans
- PUT /api/loans/<id>/return
- GET /api/loans/stats

Le Loans Service est le plus complexe : il orchestre les deux autres services. Lors d'un nouvel emprunt, il vérifie la disponibilité via le Books Service et valide l'utilisateur via le Users Service. La durée standard est de 14 jours. Il calcule automatiquement les retards (overdue) et notifie l'état de chaque prêt. Le retour d'un livre déclenche automatiquement l'incrémentation du stock disponible.

## Récapitulatif des endpoints API

| Service | Méthode | Endpoint | Description |
|---|---|---|---|
| Books | GET | /api/books | Liste tous les livres (filtre titre/auteur/ISBN) |
| Books | POST | /api/books | Ajouter un livre au catalogue |
| Books | PUT | /api/books/<id>/availability | Mettre à jour la disponibilité |
| Books | GET | /api/books/stats | Statistiques globales du catalogue |
| Users | GET | /api/users | Liste des membres (filtre par type) |
| Users | POST | /api/users/login | Authentification utilisateur |
| Users | GET | /api/users/stats | Répartition par type d'utilisateur |
| Loans | POST | /api/loans | Enregistrer un emprunt |
| Loans | PUT | /api/loans/<id>/return | Retourner un livre |
| Loans | GET | /api/loans/stats | Statistiques (actifs, retard, retournés) |

# 04 — Pipeline CI/CD (Jenkins)

Le pipeline CI/CD est défini dans le fichier **Jenkinsfile** à la racine du projet. Il automatise l'ensemble du cycle de vie : depuis la récupération du code source jusqu'au déploiement en production, en passant par l'analyse statique, les tests unitaires, la construction des images Docker et la vérification de santé des services.

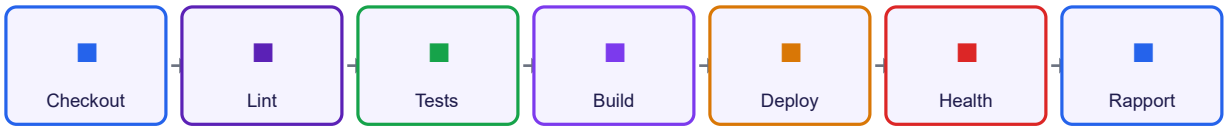



Figure 2 — Étapes du pipeline Jenkins

## Étapes du pipeline en détail

**■ Étape 1 — Checkout**

Récupération du code source depuis le dépôt GitHub via git checkout scm. Les 5 derniers commits sont affichés dans les logs pour traçabilité.

**■ Étape 2 — Analyse du code (Lint)**

Exécution en parallèle de flake8 sur les trois services Python (books, users, loans). La longueur maximale de ligne est fixée à 120 caractères. Cette étape est non bloquante pour ne pas interrompre le pipeline en cas d'avertissements mineurs.

**■ Étape 3 — Tests unitaires**

Exécution en parallèle de pytest sur chaque service. Les dépendances sont installées via pip depuis les fichiers requirements.txt. Le pipeline continue même sans tests définis (comportement gracieux).

**■ Étape 4 — Build Docker Images**

Construction des 4 images Docker (books-service, users-service, loans-service, frontend). Chaque image est taguée avec le numéro de build Jenkins et le tag :latest pour faciliter les rollbacks.

**■ Étape 5 — Déploiement**

Arrêt de l'ancienne version (docker-compose down) puis démarrage de la nouvelle version avec reconstruction des images. Une attente de 20 secondes laisse le temps aux services de s'initialiser.

**■ Étape 6 — Vérification de santé**

Interrogation du endpoint /health de chaque microservice (5 tentatives, toutes les 5 secondes). Cette vérification confirme que les services sont opérationnels avant de clore le pipeline.

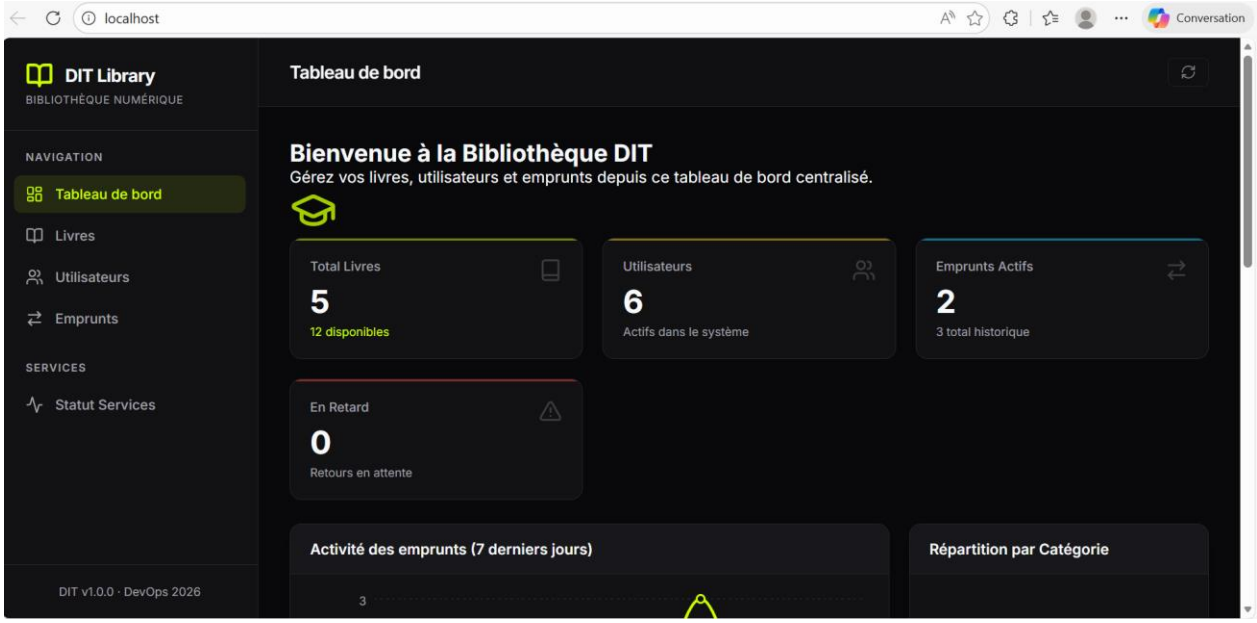
**■ Étape 7 — Rapport de déploiement**

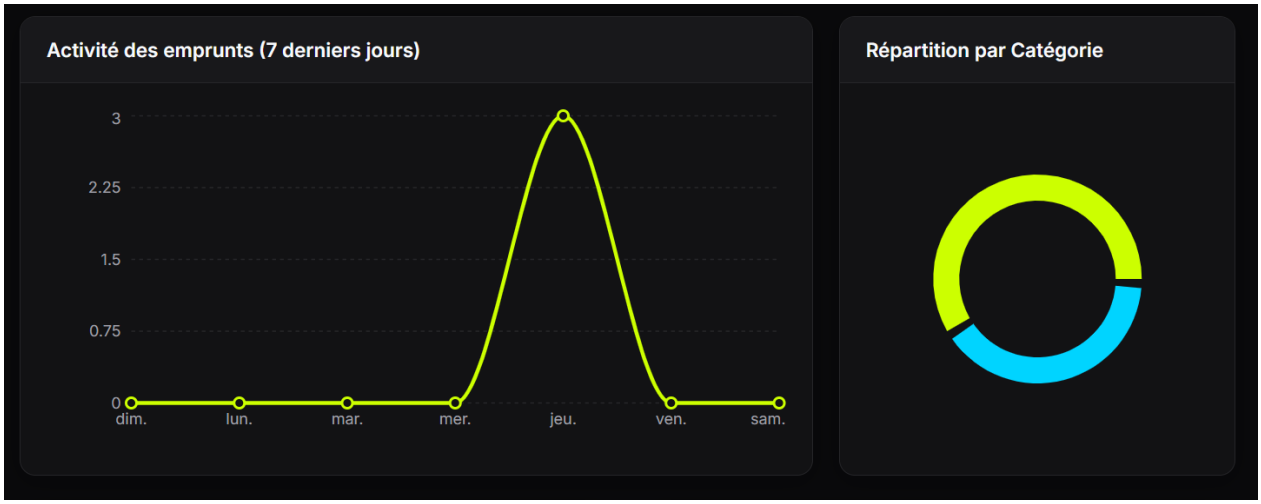
Génération d'un rapport console avec le numéro de build, la date, la branche Git et l'état de tous les conteneurs (docker ps). Les hooks post-pipeline gèrent les messages de succès/échec et le nettoyage des images orphelines.

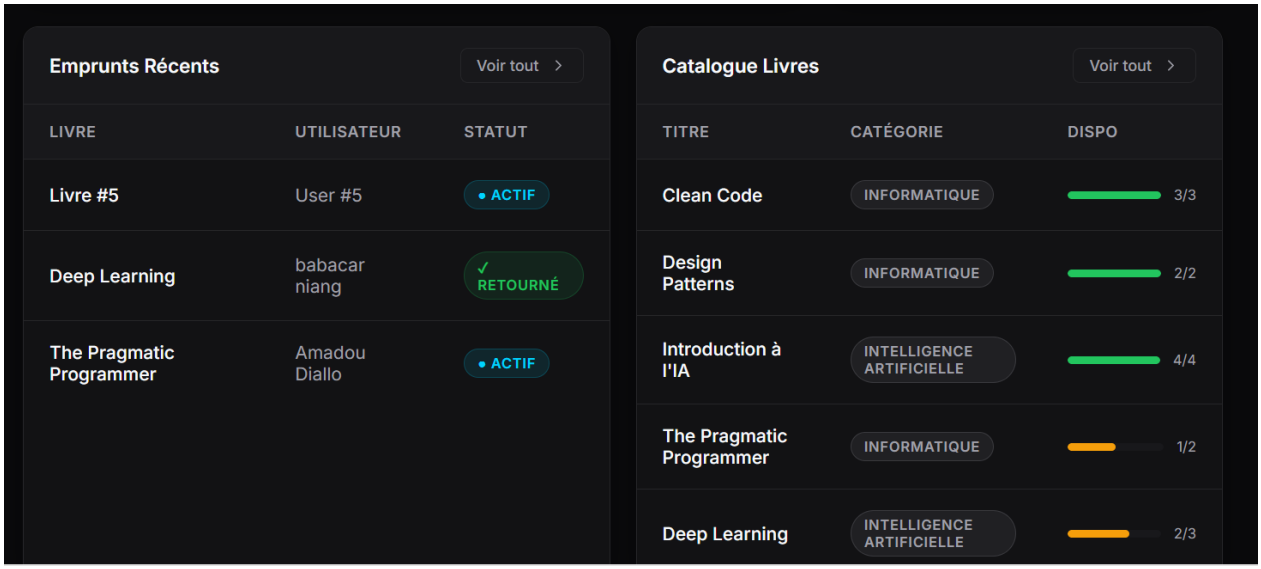
## Configuration Jenkins

| Paramètre | Valeur |
|---|---|
| Nom du projet | dit-bibliotheque |
| Registry Docker | ghcr.io/dit-devops |
| Timeout global | 30 minutes |
| Builds conservés | 10 derniers |
| Exécution parallèle | Lint et Tests (3 stages simultanés) |
| Agent | any (Jenkins master ou agent disponible) |

# 05 — Captures d'écran de l'Application






# 06 — Conclusion

La DIT Bibliothèque Numérique illustre concrètement l'application des pratiques DevOps modernes dans un contexte académique réaliste. Le projet démontre la complémentarité des outils : microservices pour la modularité, Docker pour la portabilité, Jenkins pour l'automatisation et React pour l'expérience utilisateur.

## Points forts du projet

- Architecture découplée facilitant la maintenance et l'évolution indépendante de chaque service
- Pipeline CI/CD complet avec parallélisation des étapes de lint et de tests
- Documentation API automatique via Swagger/Flasgger sur chaque microservice
- Interface React moderne avec graphiques en temps réel (Recharts) et mode sombre
- Health checks Docker natifs garantissant l'ordre de démarrage des services
- Communication inter-services orchestrée par le Loans Service (pattern Orchestrator)

## Améliorations possibles

- Ajouter une authentification JWT et un API Gateway centralisé
- Mettre en place des tests d'intégration et de charge (Locust, k6)
- Déploiement sur un cluster Kubernetes pour la mise à l'échelle horizontale
- Intégration d'un système de cache Redis pour les endpoints les plus sollicités